

Online Robot Parameter Tuning from Demonstration and Multi-Modal Context Classification

Ryan Lee¹, Justin Hart¹, Robert Blake Anderson¹, Corrie Van Sice¹, Andrew Sharp¹, Mitch Pryor¹

Abstract—Configuring autonomous navigation parameters is a labor-intensive, expert-driven process, and offline tuning limits a robot’s ability to adapt to varying environmental conditions. Adaptive Planner Parameter Learning from Demonstration (APPLD) learns multiple planner parameter sets from a single demonstration, each tied to a distinct environmental “context.” Our work advances both the sensing and inference pipelines in APPLD. For sensing, we replace APPLD’s planar LiDAR input with a semantic occupancy grid fusing RGB segmentation and 3D LiDAR, consumed by a CNN context classifier. In context inference, hard one-hot encoding for parameter selection is replaced by entropy-weighted interpolation across the top-ranked candidates, smoothing parameter transitions. The approach—Semantic APPLD (S-APPLD)—is validated on a physical robot operating in an unstructured outdoor environment with vehicle and pedestrian traffic.

I. INTRODUCTION

Configuring an autonomous navigation stack for a new environment is laborious and brittle. High-level parameters—maximum velocity, obstacle inflation radius, trajectory optimization weights—strongly shape behavior, and a set suited to a rural field may produce erratic motion in a narrow corridor or urban plaza. Tuning typically requires expert trial-and-error, and the resulting static configuration rarely performs well across changing conditions. Adaptive Planner Parameter Learning from Demonstration (APPLD) [2] addresses both issues at once: from a single teleoperated demonstration by a non-expert operator, it segments the run with the Bayesian changepoint algorithm CHAMP [3], fits each segment’s parameters via CMA-ES [4], and switches among the resulting sets online via a lightweight classifier trained on planar LiDAR statistics.

Our work targets two limitations of APPLD we have encountered in field application: low context-class separability in geometrically similar environments, and chattering in parameter values during context transitions. To address the former, we enrich the context-sensing method by projecting visually detected semantic objects into a top-down occupancy grid that serves as input to a CNN context classifier, distinguishing contexts that are geometrically similar yet semantically distinct (e.g., a parking lot populated by people

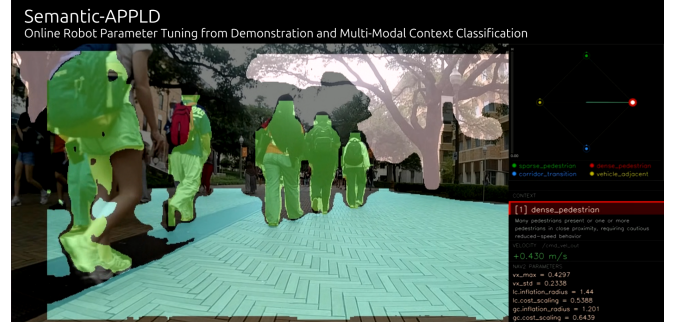


Fig. 1: S-APPLD in operation. *Left*: segmentation of pedestrians (green), walkway (cyan), and trees (white) feeding the occupancy grid input to the context classifier. *Right*: context confidences, current classification (*dense_pedestrian*), commanded velocity, and live Nav2 parameters—reduced here for the pedestrians ahead.

vs. traffic cones). Second, we replace APPLD’s one-hot context selection—a source of high-frequency switching at context boundaries—with an entropy-based interpolation that blends the top-scoring contexts’ parameter sets by classifier confidence, yielding smooth transitions. We demonstrate the enhanced system, Semantic APPLD (S-APPLD), on a physical robot navigating an unstructured outdoor environment with vehicle and pedestrian traffic (Fig. 1).

S-APPLD extends the APPL paradigm [5], [6], [7], [8], [9]. Context-dependent parameter selection outside the APPL line uses offline per-context search [10], RL-based hard switching [11], or scene-aware module switching [12]—none blends parameters continuously—while tuning guides and classical search [13], [14], [15], [16], [17] yield a single static set. End-to-end policies [18] trade away the interpretability of a classical stack; S-APPLD treats the planner as a black box.

II. METHOD

S-APPLD modifies two components of APPLD: a CNN consuming a top-down semantic occupancy grid supersedes the planar-LiDAR classifier (Sec. II-A), and entropy-based interpolation over the top-scoring contexts supersedes one-hot context selection (Sec. II-C). Sec. II-B describes how each context’s parameter set $\theta_{(i)}$ is fit; these are the sets blended in Sec. II-C.

A. Context Classification

The S-APPLD classifier consumes a top-down semantic occupancy grid in which each cell carries the class of the

*This material is based upon work supported by the University Technology Development Directorate (UTDD), United States Army Transformation and Training Command (T2COM), under Contract No. W911NF-24-C-0091. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Army Futures Command, and no such endorsement should be inferred. DISTRIBUTION STATEMENT A. Approved for public release; distribution is unlimited. OPSEC #11118

¹Department of Mechanical Engineering, University of Texas at Austin, Austin, TX 78712, USA

object detected there. A CNN trained on labeled examples maps this grid to logits over the trained contexts, driving the interpolation in Sec. II-C.

a) *Input construction.*: Given a calibrated camera, 3D LiDAR, and a YOLOv12 segmentation model [19] trained on the classes of interest, the input is built in six stages: (1) the segmentation model produces a per-pixel classmap from the RGB image; (2) the LiDAR point cloud is projected into the camera frame via the calibrated pinhole model; (3) each projected point inherits the class of its associated pixel, with class 0 reserved for points outside the camera frustum or with no detection; (4) the point cloud is voxelized by dividing by the voxel width and integer-rounding, with duplicates per voxel removed; (5) voxels are orthographically projected to a top-down classmap by dropping the z index, with collisions resolved by the last class encountered; and (6) the classmap is one-hot encoded to an $n \times m \times m$ tensor (n classes, $m \times m$ grid).

b) *Classifier training.*: Class labels can be obtained by changepoint detection [3] as in APPLD, or by hand labeling as done here. An Optuna [20] TPE [21] search over architecture and training hyperparameters selects the final model: a compact CNN with a 3×3 convolutional stem, a residual block [22] with Squeeze-and-Excitation attention [23] and GroupNorm [24], a non-residual convolutional block, and a two-layer MLP head.

B. Per-Context Parameter Optimization

Each context is characterized by a set of parameters for Nav2 [1], the de facto navigation framework for ROS 2, which we run with the Smac lattice global planner and the Model Predictive Path Integral (MPPI) local controller. From the stack’s many exposed parameters we tune the six that most prominently shape the behaviors separating our contexts: the inflation radius and cost scaling factor of the global and local costmaps, together with the controller’s maximum velocity and velocity standard deviation (Table I). For a given context, CMA-ES searches over parameter vectors θ to minimize how far Nav2’s behavior diverges from the demonstrated segment: each candidate θ is evaluated by replanning the segment, forward-integrating the commanded velocities through a unicycle model, and comparing the resulting trajectory to the recording via a weighted composite cost,

$$\mathcal{L}(\theta) = w_{pos} \text{RMSE}_{pos} + w_{yaw} \text{RMSE}_{yaw} + w_v \text{RMSE}_v + w_w \text{RMSE}_w + w_d e_{dur}, \quad (1)$$

where RMSE_{pos} and RMSE_{yaw} are RMS position and (wrapped) heading errors against the demonstrated path, RMSE_v and RMSE_w are RMS errors between commanded and demonstrated linear and angular velocities, and $e_{dur} = |N_{cmd} - N_{demo}| / N_{demo}$ penalizes duration mismatch. Weights ($w_{pos}=2$, $w_{yaw}=1$, $w_v=5$, $w_w=1$, $w_d=2$) emphasize linear-velocity tracking, the primary behavioral signature of each context.

TABLE I: Nav2 parameters for the PH2 static baseline and the per-context values optimized by CMA-ES, with the optimizer’s best cost (RMSE) per context. Contexts: Sparse Ped. (sparse pedestrian), Dense Ped. (dense pedestrian), Corridor Trans. (corridor transition), Vehicle Adj. (vehicle adjacent).

Scope	Parameter (range)	Baseline	Sparse Ped.	Dense Ped.	Corridor Trans.	Vehicle Adj.
<i>RMSE</i>		—	3.50	2.91	1.71	4.24
Global Costmap	Inflation Radius (m) (1.2–5.0)	2.00	1.45	1.20	1.72	1.45
	Cost Scaling Factor (0.5–5.0)	3.50	1.97	0.64	1.76	1.99
Local Costmap	Inflation Radius (m) (1.2–5.0)	2.00	1.46	1.44	1.79	1.65
	Cost Scaling Factor (0.5–5.0)	3.50	2.23	0.54	1.75	1.71
Controller	Max Velocity (m/s) (0.4–2.0)	1.60	1.99	0.43	0.87	1.20
	Velocity Std Dev (m/s) (0.25–3.0)	0.90	1.00	0.23	0.60	1.00

Table I lists the resulting per-context values, the static baseline, and CMA-ES’s best cost; these are the sets $\theta_{(i)}$ used in the course trials (Sec. III-A) and blended in Sec. II-C. Max-velocity values are upper bounds on commanded velocity, not targets; because the deployed bound is itself an interpolated blend (Sec. II-C), segment averages can modestly exceed a context’s nominal maximum.

C. Parameter Interpolation from Entropy of Logits

APPLD’s hard single-candidate selection makes parameters jump discontinuously whenever the active context changes. We instead interpolate over the top-scoring contexts, with an entropy criterion setting how many contribute.

Let $\sigma(z)$ denote the softmax of the classifier’s logit vector z , with entries sorted $\sigma_{(1)} \geq \dots \geq \sigma_{(K)}$ over K trained contexts (Algorithm 1). For each candidate $k \in \{2, \dots, K\}$, we renormalize the top entries, $\hat{\sigma}_i^{(k)} = \sigma_{(i)} / \sum_{j=1}^k \sigma_{(j)}$, and compute the entropy ratio $p_k = H(\hat{\sigma}^{(k)}) / H(\sigma(z))$. We select the smallest k with $p_k \geq T$ for a threshold $T \in (0, 1]$; $k = 1$ is excluded since its renormalized distribution has zero entropy and never satisfies the threshold. The final parameter vector is $\sum_{i=1}^k \hat{\sigma}_i^{(k)} \theta_{(i)}$, where $\theta_{(i)}$ is the fitted parameter set of the i -th-ranked context (Sec. II-B).

Algorithm 1 Entropy-Based Top- k Context Selection

Input: logit vector z , threshold $T \in (0, 1]$
 $\sigma \leftarrow \text{softmax}(z)$
Sort σ in descending order: $\sigma_{(1)} \geq \dots \geq \sigma_{(K)}$
 $H_{\text{full}} \leftarrow -\sum_{i=1}^K \sigma_{(i)} \log \sigma_{(i)}$
for $k = 2, 3, \dots, K$ **do**
 $\hat{\sigma}_i^{(k)} \leftarrow \sigma_{(i)} / \sum_{j=1}^k \sigma_{(j)}$
 $H_k \leftarrow -\sum_{i=1}^k \hat{\sigma}_i^{(k)} \log \hat{\sigma}_i^{(k)}$
if $H_k / H_{\text{full}} \geq T$ **then**
 return k
end if
end for
return K

A confident prediction keeps its dominant weight after renormalization, so the blend stays near-one-hot (but never exactly one-hot) regardless of k ; an ambiguous prediction yields a genuine multi-context blend. Restricting to the top k keeps inapplicable, never-validated contexts out of the

blend. The threshold T mainly controls admission of low-probability contexts and has limited effect in the confident regime; we use $T = 0.95$ in all experiments.

III. RESULTS

We evaluate S-APPLD on a Husarion Panther UGV with an Ouster OS1 LiDAR, Luxonis Oak-D camera, and a Nuvo-9160GC auxiliary computer, running a ROS 2 Humble stack (Nav2 with the Smac global planner and MPPI local controller). Localization fuses LiDAR and IMU data via an EKF (`robot_localization`) with AMCL on a map pre-built using `slam_toolbox`. The experimental areas span distinct object populations and LiDAR profiles, with pedestrian and vehicle density varying by time of day.

The pipeline of Sec. II-A is instantiated with a voxel width of 0.05 m over a 512×512 grid, a vertical column filter of -0.5 – 2.0 m, and Oak-D intrinsics and LiDAR-to-camera extrinsics fixed by calibration; the resulting $n_{\text{classes}}+1$ -channel tensor is stored at float16 precision and passed to a compact residual CNN trained with AdamW ($\text{lr} = 5.1 \times 10^{-4}$) under label smoothing and focal loss. At deployment, parameter dispatch to Nav2 is gated by a cosine-similarity threshold on the logit vector ($\tau_{\text{cos}} = 0.8$), rate-limited to 1 Hz, with the ensemble interpolation budget capped at 95% cumulative mass (Sec. II-C).

A. Experimental Design

a) Course experiment (PH1–PH3).: Phase 1 (PH1) comprises teleoperated data collections on routes separate from the course. Recordings across four contexts—sparse pedestrian, dense pedestrian, corridor transition, and vehicle adjacent—train the classifier (20/5 train/validation split). A varied-driving dataset, with speed and obstacle clearance deliberately modulated per context, provides the segments for parameter fitting (Sec. II-B) and trains the YOLO segmentation model. A single demonstration along the course itself serves only as a human baseline. The robot then traverses the course—spanning a pedestrian mall, an asphalt street, and a parking lot—11 times with a static Nav2 configuration (PH2) and 11 times with S-APPLD engaged (PH3), using the per-context parameters of Table I.

b) Pedestrian-density generalization (PH4).: To evaluate the dense pedestrian context over a representative sample, we hold location fixed on a single walkway and vary pedestrian density across runs (sparse: $n = 6$; dense: $n = 5$).

c) Inference-rule replay.: To isolate the contribution of the new selection rule, we replay the recorded PH3 runs with classifier and sensor inputs fixed. Interpolation updates are the dispatches recorded during deployment; one-hot updates reconstruct the same classifier stream through APPLD-style mode-filtered hard selection (a five-sample mode filter [2]) under the same dispatch rate limit. Parameter vectors are min–max normalized per dimension over the per-context optimized sets before computing the update magnitude $\|\Delta\pi\|$. Updates are binned by classifier confidence into equal-size entropy terciles; a discontinuity threshold $\tau = 0.812$ —half the median inter-context parameter distance—marks context-swap-scale updates.

B. Course Experiment Results (PH1–PH3)

Table II reports mean speed per context (PH2) and mean accuracy and speed (PH3). The static baseline produces near-uniform speed across all four contexts (1.4–1.5 m/s), reflecting its fixed parameters. S-APPLD identified the correct context with 91.12% overall accuracy, and segment speeds followed the optimized parameters: slowest in dense pedestrian traffic, fastest where foot traffic was light, and reduced in the corridor transition and vehicle-adjacent contexts. Dense pedestrian was the lowest and most variable ($76.29 \pm 10.22\%$), but the context covered only 2.53% of the course. Its statistic is therefore dominated by entry and exit boundary frames, where the semantic grid is a transient class mixture; PH4 measures steady-state performance free of this effect (Sec. III-C). Both configurations completed every run, and Fig. 2 shows that S-APPLD tracks the PH1 human demonstration more closely than the static baseline throughout the course.

C. Pedestrian-Density Results (PH4)

S-APPLD reliably identified both PH4 contexts and drove markedly differently between them (Table III): sparse runs averaged roughly $3.9\times$ the speed of dense runs (1.73 vs. 0.45 m/s), confirming adaptation is driven by observed scene conditions rather than location alone.

D. Inference-Rule Analysis

In the replay study, large updates were the norm for one-hot and the exception for interpolation under ambiguity (Fig. 3). The one-hot median rose from 0.89 to 1.43 across terciles, climbing above τ and spiking further in the most uncertain tercile. Interpolation stayed flat-to-declining ($0.84 \rightarrow 0.84 \rightarrow 0.74$) and remained below τ throughout—roughly half the one-hot median in that tercile. The interquartile

TABLE II: Mean $\pm$ std. dev. of linear speed and context classification accuracy across 11 runs, by scenario, for PH2 (Nav2 baseline) and PH3 (S-APPLD). Accuracy (Acc.) compares predicted context against manually labeled ground truth and is marked *n/a* for the unclassified baseline. Total distance traveled: 342.44 ± 2.14 m (PH2), 343.78 ± 1.00 m (PH3). PH3 overall classification accuracy: $91.12 \pm 1.17\%$.

Phase	Sparse Pedestrian		Dense Pedestrian		Corridor Transition		Vehicle Adjacent	
	Acc. (%)	Spd. (m/s)	Acc. (%)	Spd. (m/s)	Acc. (%)	Spd. (m/s)	Acc. (%)	Spd. (m/s)
PH2	<i>n/a</i>	1.49 ± 0.01	<i>n/a</i>	1.39 ± 0.10	<i>n/a</i>	1.48 ± 0.01	<i>n/a</i>	1.47 ± 0.01
PH3	93.75 ± 2.98	1.76 ± 0.02	76.29 ± 10.22	0.53 ± 0.04	92.05 ± 2.93	0.80 ± 0.01	90.54 ± 3.34	1.00 ± 0.01

ranges separate similarly in that tercile (0.88–1.82 for one-hot vs. 0.49–1.02 for interpolation), and interpolation’s cap on cumulative blended mass structurally precludes a full unit-weight context swap.

IV. CONCLUSIONS

We present S-APPLD, an extension of APPLD that enriches context-aware navigation parameter tuning along two axes. On the sensing side, fusing RGB-based semantic segmentation with 3D LiDAR into a top-down semantic occupancy grid lets the classifier discriminate contexts that are geometrically similar but semantically distinct. On the inference side, an entropy-based top- k interpolation scheme replaces APPLD’s hard one-hot switching with smooth, confidence-weighted parameter transitions. Physical-robot

experiments across four trained contexts in an unstructured outdoor environment sustained $91.12 \pm 1.17\%$ overall classification accuracy over 11 autonomous course traversals and modulated speed by up to $3.9\times$ between pedestrian densities, confirming behavior is driven by scene conditions rather than location. Under ambiguity, the entropy-weighted blend roughly halved the median parameter-update magnitude rel-

TABLE III: Mean $\pm$ std. dev. for PH4 pedestrian-walkway runs (sparse: $n = 6$; dense: $n = 5$). Acc. is vs. manually labeled ground truth.

Context	Acc. (%)	Spd. (m/s)
Sparse Pedestrian	95.08 ± 4.96	1.73 ± 0.07
Dense Pedestrian	97.52 ± 3.60	0.45 ± 0.03

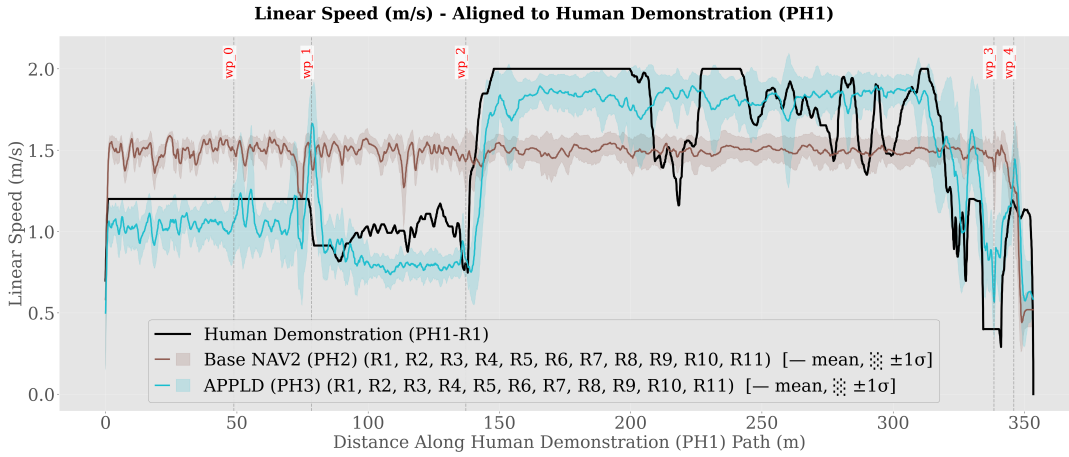


Fig. 2: Linear speed along the course: PH1 (human demonstration) and mean $\pm 1\sigma$ across 11 runs for PH2 (baseline) and PH3 (S-APPLD). PH3 tracks the human profile more closely than PH2 throughout.

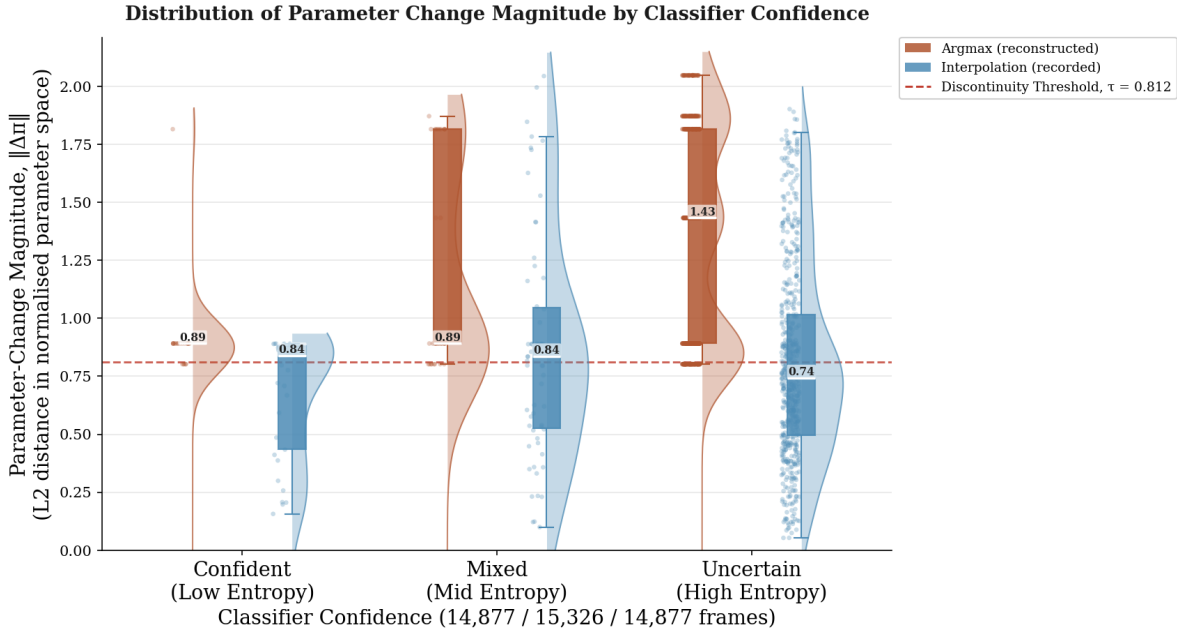


Fig. 3: Parameter-change magnitude $\|\Delta\pi\|$ per update, by classifier confidence (equal-size entropy terciles). Half-violins show density, boxes the quartiles and median.

ative to one-hot selection. Neither extension has yet been ablated against the APPLD component it replaces (validated on different hardware); an ablation on a shared platform is priority future work, alongside broadening the training curriculum to diverse lighting and weather. Two further limitations remain: interpolation applies only to continuous parameters, as blending discrete ones can produce invalid values; and context sensing is camera-bounded—LiDAR returns outside the frustum carry no semantic label, and RGB detection degrades under adverse lighting or weather.

REFERENCES

- [1] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A navigation system,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2020.
- [2] X. Xiao, B. Liu, G. Warnell, J. Fink, and P. Stone, “APPLD: Adaptive planner parameter learning from demonstration,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4541–4547, 2020.
- [3] S. Niekum, S. Osentoski, C. G. Atkeson, and A. G. Barto, “CHAMP: Changepoint detection using approximate model parameters,” Carnegie Mellon University, 2014.
- [4] N. Hansen, S. D. Müller, and P. Koumoutsakos, “Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES),” *Evolutionary Computation*, vol. 11, no. 1, pp. 1–18, 2003.
- [5] X. Xiao, Z. Wang, Z. Xu, B. Liu, G. Warnell, G. Dhamankar, A. Nair, and P. Stone, “APPL: Adaptive planner parameter learning,” *Robotics and Autonomous Systems*, vol. 154, p. 104132, 2022.
- [6] Z. Xu, G. Dhamankar, A. Nair, X. Xiao, G. Warnell, B. Liu, Z. Wang, and P. Stone, “APPLR: Adaptive planner parameter learning from reinforcement,” in *2021 IEEE Int. Conf. Robotics and Automation (ICRA)*. IEEE, 2021.
- [7] L. Perille, A. Truong, X. Xiao, and P. Stone, “Benchmarking metric ground navigation,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2020.
- [8] Z. Wang, X. Xiao, B. Liu, G. Warnell, and P. Stone, “APPLI: Adaptive planner parameter learning from interventions,” in *2021 IEEE Int. Conf. Robotics and Automation (ICRA)*. IEEE, 2021.
- [9] Y. Lu, B. Wang, Z. Wu, Y. Li, X. Lin, C. Mao, and X. Xiao, “APPLV: Adaptive planner parameter learning from vision-language-action model,” *arXiv preprint arXiv:2603.08862*, 2026.
- [10] A. Binch, G. P. Das, J. P. Fentanes, and M. Hanheide, “Context dependant iterative parameter optimisation for robust robot navigation,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2020, pp. 3937–3943.
- [11] E. Goldsztejn, T. Feiner, and R. Brafman, “PTDRL: Parameter tuning using deep reinforcement learning,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2023, pp. 11356–11362.
- [12] K. Weerakoon, A. J. Sathymoorthy, M. Elnoor, and D. Manocha, “AdVENTR: Autonomous robot navigation in complex outdoor environments,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2023.
- [13] L. Ward, C. Snyder, and A. Nagchaudhuri, “Exploring ROS2 based robot navigation: Parameter tuning using the NAV2 package,” in *Proc. ASME Int. Design Engineering Technical Conf. and Computers and Information in Engineering Conf. (IDETC-CIE)*, vol. 5, 2025, Paper V005T07A024.
- [14] K. Zheng, “ROS navigation tuning guide,” *arXiv preprint arXiv:1706.09068*, 2019.
- [15] D. Mlinarček, R. Jánoš, M. Málík, J. Svetlík, J. Semjon, and Š. Ondočko, “Costmap tuning for autonomous navigation: A simulation and real-world study on the Hiwonder JetAcker,” *Applied Sciences*, vol. 16, no. 4, p. 1923, 2026.
- [16] Y. Liang, F. Hong, Q. Lin, S. Bi, and L. Feng, “Optimization of robot path planning parameters based on genetic algorithm,” in *Proc. IEEE Int. Conf. Real-time Computing and Robotics (RCAR)*, 2017, pp. 529–534.
- [17] D. Stenger, M. Nitsch, and D. Abel, “Joint constrained Bayesian optimization of planning, guidance, control, and state estimation of an autonomous underwater vehicle,” *arXiv preprint arXiv:2205.14669*, 2022.
- [18] J. Cheng, Y. Chen, and Q. Chen, “PLUTO: Pushing the limit of imitation learning-based planning for autonomous driving,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14327>
- [19] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” in *Advances in Neural Information Processing Systems*, 2025.
- [20] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*. New York, NY, USA: ACM, 2019, pp. 2623–2631.
- [21] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger, Eds., vol. 24, 2011.
- [22] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [23] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [24] Y. Wu and K. He, “Group normalization,” in *Proc. European Conf. Computer Vision (ECCV)*, Munich, Germany, Sept. 2018, pp. 3–19.